

Viva Questions & Simple Spoken Answers

Written in simple, natural language so you can explain Frontend, Backend, Database, and UI/UX confidently to your Professor

FRONTEND

HTML5, CSS3, Vanilla
JavaScript

BACKEND

Supabase API, Node.js, Live
Updates

DATABASE

PostgreSQL (12 Tables) + RLS
Security

UI / UX THEME

MIT Burgundy (#750014) &
Cream (#F8F7F4)

HOW TO INTRODUCE YOUR PROJECT IN 30 SECONDS

- "Sir/Ma'am, we built the **MIT Academic Portal**—a full-stack web application for Students and Faculty."
- "Students can register, check their subject-wise attendance, view marks/SGPA, read notices, and check their weekly timetable."
- "Faculty can manage the student directory, mark daily attendance, enter exam marks, publish notices, and edit the timetable."
- "All data is stored in a shared **Supabase PostgreSQL database** so any update made by Faculty appears immediately on the Student's device."

5 MAIN FILES IN YOUR PROJECT (IF ASKED TO SHOW CODE)

- `index.html` — Contains the webpage design (HTML/CSS) and screen logic for Student and Faculty dashboards.
- `js/supabase-client.js` — Connects the browser to the backend server for Login, Database calls, and Live updates.
- `js/portal-api.js` — Contains main functions like Student Registration, Attendance saving, and SGPA calculation.
- `server.js` — The backend server that handles Login security, Database queries, and Real-time notifications.
- `supabase/migrations/` — Contains the SQL files that create the 12 database tables and security rules.

Section 1: Project Overview & Architecture

QUESTIONS 1 – 6

Q1 Explain the architecture of your project in simple words. How does it work?

ARCHITECTURE

Simple Answer to Speak: "Sir, our project uses a **3-layer (3-Tier) architecture**:

1. **Frontend (User Interface):** Built with HTML, CSS, and JavaScript. This is what the Student and Faculty see in their browser.
2. **Backend (API Layer):** Uses Supabase Auth and REST API. When a user clicks a button, the frontend sends a request to the backend server.
3. **Database Layer:** A PostgreSQL relational database with 12 tables where all students, attendance, marks, notices, and timetables are permanently saved."

Flow to remember: Browser Screen (index.html) → API Helper (portal-api.js) → Backend Server (server.js) → Database

Q2 Why did you move from `localStorage` to a Supabase Database? What was the problem earlier?

MAIN PROBLEM SOLVED

Simple Answer to Speak: "Earlier, when the project used `localStorage`, data was saved only inside that single browser on that single phone or laptop. For example, if a student registered on their phone, the Faculty logging in on a computer could not see that new student at all! Also, anyone could open browser settings and change their own marks.

We fixed this by connecting the app to a **shared Supabase database**. Now, whether a student registers from a phone or laptop, their record goes to one central database and immediately appears on the Faculty's laptop."

Q3 What do you still store in `localStorage` now?

STORAGE

Simple Answer to Speak: "We do NOT store any student data, marks, or attendance in `localStorage` anymore. We only store two small things:

- The **Login Session Token** (so the user stays logged in when they refresh the page).
- The **Remember Me preference** on the login screen."

Q4 Why did you split your JavaScript code into `index.html`, `portal-api.js`, and `supabase-client.js` ?

CODE ORGANIZATION

Simple Answer to Speak: "To keep the code clean and easy to maintain, we divided the work into three parts:

- `index.html` only handles showing buttons, tables, and forms on the screen.
- `portal-api.js` handles college rules—like calculating SGPA, registering a student, or saving attendance.
- `supabase-client.js` handles talking to the server over the network."

Q5 Why doesn't GitHub Pages (`github.io`) run a backend server by itself?

HOSTING

Simple Answer to Speak: "GitHub Pages is a **static hosting service**—it can only show static files like HTML, CSS, and JS, but it cannot run a database or Node.js server. To share data across devices, the webpage connects to our running backend server or a cloud Supabase URL."

Q6 When a new student registers, why are their SGPA, CGPA, and Attendance shown as 0 instead of sample numbers?

REAL DATA LOGIC

Simple Answer to Speak: "Because in a real college portal, a newly admitted student hasn't attended any lectures or taken any exams yet. Giving them fake marks or fake 8.5 CGPA on registration would be wrong. So new students start with `0` attendance and `0.00` SGPA until a Faculty member actually marks their attendance and enters their grades."

Q7 Why did you use normal HTML, CSS, and Vanilla JavaScript instead of React or Angular?

FRONTEND TECH

Simple Answer to Speak: "Using pure HTML, CSS, and Vanilla JavaScript makes the portal **very fast and lightweight**:

- It runs directly in the browser without needing heavy build tools or external libraries.
- The page loads almost instantly.
- We have direct control over how every table, modal, and form updates on the screen."

Q8 How does your website switch between Login, Student Dashboard, and Faculty Dashboard without reloading the page?

SINGLE PAGE APP

Simple Answer to Speak: "Our portal works as a **Single-Page Application (SPA)** using URL hashes like `#login`, `#student-dashboard`, and `#faculty-dashboard`. When a user logs in or clicks a sidebar tab, JavaScript hides the old section and shows the new section using CSS classes, while fetching fresh data from the database in the background."

Q9 What is `async / await` in JavaScript, and where did you use it?

JAVASCRIPT

Simple Answer to Speak: "When our webpage asks the database for data (like loading the Student Directory), it takes a few milliseconds over the network. `async / await` lets JavaScript wait for the database reply **without freezing the screen**.

For example, in `renderFacultyStudentList()`, we first show 'Loading student directory...', then `await getFacultyStudents()` from the database, and once the data arrives, we display the student rows in the table."

Q10 How do you protect the website if someone types HTML or `<script>` tags inside the Student Name or Notice box?

XSS SECURITY

Simple Answer to Speak: "That attack is called **Cross-Site Scripting (XSS)**. To prevent it, we pass user text through our `escapeHtml()` function before showing it on the page. It turns special symbols like `<` and `>` into plain text so the browser never runs them as code."

Q11 What are CSS Variables (`--mit-red`, etc.) and why did you use them?

CSS STYLING

Simple Answer to Speak: "CSS Variables let us save our main theme colors and fonts in one place at the top of the CSS file (inside `:root`), such as `--mit-red: #750014` and `--bg-cream: #F8F7F4`. Instead of typing color codes hundreds of times, we reuse the variable name everywhere, keeping the design consistent."

Q12 How does the search bar and filter work in the Faculty Student Directory?

SEARCH & FILTER

Simple Answer to Speak: "First, `getFacultyStudents()` loads the real student list from the database. Then, when the Faculty types in the search box or selects a Semester/Department dropdown, JavaScript filters the list by checking if the student's Name, Roll Number (`STU2026001`), or Department matches the search text, and immediately updates the table."

Q13 What happens when a logged-in user presses Refresh (F5) in the browser?

SESSION RESTORE

Simple Answer to Speak: "The user does not get logged out. When the page reloads, JavaScript checks the saved login token, verifies it with the backend server, checks from the `profiles` table whether the user is a Student or Faculty, loads their latest data from the database, and keeps them on their dashboard."

Q14 How did you make the portal work on both mobile screens and laptop screens?

RESPONSIVE DESIGN

Simple Answer to Speak: "We used **Responsive CSS Grid, Flexbox, and Media Queries** (`@media`). On a wide laptop screen, cards appear in 3 or 4 columns side-by-side. On a smaller mobile screen, they automatically stack into 1 column and tables become horizontally scrollable so nothing breaks."

Section 3: Backend, API & Real-Time Sync Questions

QUESTIONS 15 – 22

Q15 Step by step, what happens in the backend when a new student registers?

REGISTRATION FLOW

Simple Answer to Speak: "When a student fills the registration form and clicks Submit, 4 steps happen in order:

1. **Create Login Account:** The backend checks that the email isn't already used, encrypts (hashes) the password, and creates an auth account.
2. **Create Profile Row:** It adds a row in the `profiles` table with the student's name, email, and `role = 'student'`.
3. **Create Student Academic Row:** It adds a row in the `students` table with their Roll Number (like `STU2026003`), course, semester, and division.
4. **Assign Subjects & Notify Faculty:** It links the student to their semester subjects in `student_subjects` and sends a live signal so any open Faculty screen updates immediately."

Q16 How do you store passwords safely in the database? Do you save plain text passwords?

PASSWORD SECURITY

Simple Answer to Speak: "No Sir, we never store plain text passwords. If a database is ever leaked, plain passwords would expose every user. Instead, we use **PBKDF2 with SHA-256 hashing and a random Salt** (10,000 rounds). This converts the password into an irreversible scrambled string before saving it."

Q17 What if a Student selects the 'Faculty' tab on the login page and tries to log in with their student email?

ROLE SECURITY

Simple Answer to Speak: "The system will block them! We never trust the role button clicked on the frontend. After verifying the password, the backend checks the user's actual `role` column inside the database `profiles` table. If the database says `student`, they cannot enter the Faculty portal."

Q18 How does the Faculty screen update automatically when a student registers on another computer?

REAL-TIME SYNC

Simple Answer to Speak: "We used **Supabase Realtime (Server-Sent Events / SSE)**. The Faculty browser keeps a live listener open with the server. The moment a new student row is inserted into the database, the server broadcasts a message saying *'students table changed'*, and the Faculty dashboard automatically re-fetches and displays the new student."

Q19 What are the main HTTP methods (`GET`, `POST`, `PATCH`, `DELETE`) used in your project?

REST API

Simple Answer to Speak:

- `GET` — Used to **read/fetch** data (like loading the student list, notices, or timetable).
- `POST` — Used to **create/insert** new data (like registering a new student, saving attendance, or posting a notice).
- `PATCH` — Used to **update** existing data (like updating a student's phone number or SGPA).
- `DELETE` — Used to **remove** data (like deleting a notice or clearing a timetable slot).

Q20 What do HTTP Status Codes like 200, 201, 401, and 403 mean in your backend?

STATUS CODES

Simple Answer to Speak:

- `200 OK` — Request succeeded (data fetched or updated).
- `201 Created` — New record was successfully inserted into the database.
- `400 Bad Request` — Missing or invalid input data.
- `401 Unauthorized` — User is not logged in or password/token is wrong.
- `403 Forbidden` — User is logged in, but not allowed to do that action (for example, a student trying to edit marks).

Q21 How does the system calculate a student's Grade and SGPA when Faculty enters marks?

SGPA CALCULATION

Simple Answer to Speak: "For each subject, we add **Internal (out of 20) + Mid-Sem (out of 30) + End-Sem (out of 50) = Total out of 100**.

- Based on total marks out of 100, the system assigns a Letter Grade and Grade Point: **O (10 pts for ≥ 85), A+ (9 pts for ≥ 75), A (8 pts for ≥ 65), down to F (0 pts for < 40).**
- Then **SGPA** is calculated by multiplying each subject's Credits by its Grade Points, adding them up, and dividing by Total Credits: $SGPA = \frac{\sum(Credits \times Grade\ Points)}{\sum(Credits)}$."

Q22 How does Attendance percentage get calculated when Faculty marks attendance?

ATTENDANCE CALCULATION

Simple Answer to Speak: "Every time a Faculty takes attendance for a subject, the server creates a session in `attendance_sessions` and saves each student's status (`present` or `absent`) in `attendance_records` . Then the student's percentage is simply: $(Total\ Classes\ Present \div Total\ Classes\ Conducted) \times 100$."

Section 4: Database Design (PostgreSQL) & Security (RLS)

QUESTIONS 23 – 29

Table Name	Main Keys	What It Stores (In Simple Words)
departments	PK: <code>id</code> , Unique: <code>code</code>	List of college departments (e.g., Computer Engineering <code>COMP</code>).
courses	PK: <code>id</code> , FK: <code>department_id</code>	List of degree courses (e.g., B.Tech Computer Engineering).
profiles	PK: <code>id</code> , FK: <code>auth_user_id</code>	Common user details for everyone: Name, Email, Phone, and Role (<code>student</code> or <code>faculty</code>).
students	PK: <code>id</code> , FK: <code>profile_id</code>	Student-specific details: Roll No (<code>STU2026001</code>), Semester, Division, SGPA, CGPA.
faculty	PK: <code>id</code> , FK: <code>profile_id</code>	Faculty-specific details: Employee ID (<code>FAC2019042</code>), Designation, Qualification.
subjects	PK: <code>id</code> , FK: <code>faculty_id</code>	All subjects (<code>CS301</code> , <code>CS302</code>) with credits, semester, and teacher assigned.
student_subjects	FK: <code>student_id</code> , <code>subject_id</code>	Connects which student is enrolled in which subject, plus their class counts.
attendance_sessions	PK: <code>id</code> , FK: <code>subject_id</code>	One row for each lecture conducted (Date, Time Slot, Subject, Topic taught).
attendance_records	FK: <code>session_id</code> , <code>student_id</code>	Stores whether each student was <code>present</code> or <code>absent</code> in that lecture.
grades	FK: <code>student_id</code> , <code>subject_id</code>	Stores Internal, Mid-Sem, End-Sem marks, Total, and Grade for each student & subject.
notices	PK: <code>id</code> , FK: <code>author_profile_id</code>	Stores official notices/announcements posted by Faculty.
timetables & timetable_entries	PK: <code>id</code> , FK: <code>timetable_id</code>	Stores the weekly class schedule and individual day/time-slot rooms & subjects.

Q23 Why did you create 12 separate tables instead of putting everything in one single table? (What is Normalization?)

NORMALIZATION (3NF)

Simple Answer to Speak: "Putting everything in one big table causes **data repetition (redundancy)** and update mistakes. Dividing data into related tables is called **Normalization (up to 3NF)**:

- **1NF:** Every column stores a single value (no comma-separated lists of subjects or marks).
- **2NF:** We use a linking table (`student_subjects`) so subject names aren't repeated for every student.
- **3NF:** Personal info is in `profiles` , department info is in `departments` , and marks are in `grades` . If a teacher's name changes in `profiles` , it updates everywhere automatically."

Q24 What is a Primary Key and a Foreign Key? Give an example from your project.

KEYS

Simple Answer to Speak:

- **Primary Key (PK):** A unique ID that identifies each row in a table (for example, `id` in the `students` table). No two rows can have the same Primary Key.
- **Foreign Key (FK):** A column in one table that points to the Primary Key of another table to link them together. For example, `profile_id` in the `students` table points to `id` in the `profiles` table.

Q25 What is Row Level Security (RLS) in simple words?

RLS SECURITY

Simple Answer to Speak: "Row Level Security (RLS) is like a **security guard inside the database** that checks every single row before showing or changing it:

- When a **Student** is logged in, RLS allows them to see **only their own** profile, their own attendance, and their own marks. They cannot see other students' marks or faculty data, and they cannot edit marks or attendance.
- When a **Faculty** member is logged in, RLS allows them to view all students in the directory and add/update attendance, marks, notices, and timetables."

Q26 Why do your tables have both a random UUID and a college ID like

UUID VS ROLL NO

`STU2026001` ?

Simple Answer to Speak: "`STU2026001` is easy for students and faculty to read and type during login. Internally, the database uses a **UUID** (a long random ID) as the Primary Key so hackers cannot guess user IDs by simply counting 1, 2, 3, 4."

Q27 What does ON DELETE CASCADE mean in your SQL schema?

FOREIGN KEYS

Simple Answer to Speak: "It means if a parent record is deleted, its linked child records are automatically cleaned up too. For example, if an attendance session is deleted, all student attendance records for that session are automatically removed so no leftover 'orphan' data remains."

Q28 How does the database stop a Faculty member from accidentally entering two grade rows for the same student in the same subject?

UNIQUE CONSTRAINT

Simple Answer to Speak: "We added a **Composite UNIQUE rule** on `(student_id, subject_id, academic_year)` in the `grades` table. If Faculty saves marks again for the same student and subject, the database updates the existing row (called an **UPSERT**) instead of creating a duplicate row."

Q29 How do you prevent SQL Injection attacks in your backend?

SQL INJECTION

Simple Answer to Speak: "We never glue user input directly into SQL query strings. Instead, we use **Parameterized Queries (Prepared Statements with ? placeholders)**. Even if someone types SQL commands into the login box, the database treats it as harmless text."

Q30 What is the difference between UI and UX, and how did you focus on both?

UI VS UX

Simple Answer to Speak:

- **UI (User Interface)** is how the app **looks**—colors, fonts, buttons, and cards. We used MIT's official Burgundy (#750014) and soft Cream (#F8F7F4) background.
- **UX (User Experience)** is how easy and smooth the app **feels to use**—for example, giving Faculty a 1-click 'Mark All Present' button so they don't have to click 60 times in class.

Q31 Why did you choose a Warm Cream background (#F8F7F4) with Burgundy (#750014) instead of bright neon colors?

COLOR CHOICE

Simple Answer to Speak: "An academic portal is used for long hours by students and professors. Pure bright white or neon colors cause eye strain. A soft warm cream background with white cards and dark burgundy accents gives a professional university look and is very comfortable to read."

Q32 What is an 'Empty State' in UI/UX, and where did you use it?

EMPTY STATES

Simple Answer to Speak: "An Empty State is what we show when there is no data yet, so the screen doesn't look broken or blank. When a new student registers, instead of showing a blank box or 0/0 NaN% , we display a helpful message: 'No attendance sessions recorded yet' and 'Marks not yet published by faculty'."

Q33 How does your UI help a student immediately know if their attendance is safe or in danger?

VISUAL FEEDBACK

Simple Answer to Speak: "We use traffic-light color coding on attendance bars and badges:

- **Green (≥75%):** Safe attendance (meets college criteria).
- **Yellow/Amber (65% – 74%):** Borderline warning.
- **Red (<65%):** Critical defaulter risk. Plus, the dashboard tells the student how many more classes they need to attend to reach 75%."

Q34 How does the user know that an action (like saving attendance or posting a notice) actually worked?

USER FEEDBACK

Simple Answer to Speak: "Every time a user performs an action, we show immediate visual feedback: a loading indicator while saving, followed by a pop-up **Toast Notification** (e.g., 'Attendance recorded successfully') and an instant update of the table on screen."

Q35 Why did you use a Monospace font for Roll Numbers (STU2026001), Subject Codes (CS301), and Marks?

TYPOGRAPHY

Simple Answer to Speak: "In a Monospace font, every letter and number has the exact same width. This makes Roll Numbers, Subject Codes, and numerical marks line up neatly in vertical columns inside tables, making it much easier for Faculty to scan."

Section 6: Tricky Professor Questions & Live Demo Guide

QUESTIONS 36 – 40

Q36 "What happens if a student opens Browser Inspect Element / Console and tries to change their marks in the database?"

TRICKY SECURITY Q

Simple Answer to Speak: "If they change the number in Inspect Element, it only changes on their screen temporarily and disappears as soon as they refresh. And if they try to send an API command from the Console to update the `grades` table, the backend server checks their role in the database, sees they are a `student`, and blocks the request with a **403 Forbidden error**."

Q37 "Can one logged-in student see another student's marks or phone number using the API?"

PRIVACY CHECK

Simple Answer to Speak: "No, Sir. Because of Row Level Security (RLS), when a student requests data from `students` or `grades`, the server automatically filters the rows using their logged-in user ID. They only receive their own row and zero rows of other students."

Q38 "What happens if the internet disconnects or the backend server is stopped while Faculty opens the Student Directory?"

ERROR HANDLING

Simple Answer to Speak: "It does not crash or show fake old data. The `try / catch` block catches the error and displays a clear message: 'Unable to load student directory. Please try again.' along with a Retry button."

Q39 "How does the Weekly Timetable feature work between Faculty and Students?"

TIMETABLE FEATURE

Simple Answer to Speak: "We store the schedule in two linked tables: `timetables` (for the semester and division) and `timetable_entries` (for each day and time slot like Monday 9:00 AM, Subject, Room, and Teacher). When Faculty edits a slot and clicks Publish, it saves to the database and immediately updates the timetable grid on every Student's dashboard."

1. **Keep 2 Browser Windows Open Side-by-Side:** Left window on Registration page, Right window logged in as Faculty (`FAC2019042`) on the *Assigned Student Directory* tab.
2. **Register a New Student on Left Window:** Fill in a new Roll No (e.g., `STU2026009`) and click Register.
3. **Point to Right Window (The 'Wow' Moment):** Show the professor that `STU2026009` **automatically appears in the Faculty Student Directory** without even refreshing the page!
4. **Mark Attendance & Grades on Right Window:** Mark `STU2026009` Present and enter their marks in Faculty portal, and show them updating live on the Student's dashboard in the Left Window.

MIT Academic Portal — Easy Viva Voce Preparation Guide

Frontend (HTML5, CSS3, Vanilla JS) • Backend (Supabase Auth, REST API, Realtime SSE) • Database (PostgreSQL & Row Level Security)